



FACULTAD DE INGENIERÍA

Sistemas multiagente

Obligatorio 2: Juegos alternados.

2026

Profesor:

Marcos Adrian Begerez Oyhenard

Martín Iturbide Noria

Alumnos:

Natalia Campiglia - 349251

Marcos Mussio - 362839

Índice

1. Introducción	4
2. Marco Teórico	5
2.1. Juegos Alternados	5
2.2. Juegos de Suma Cero	5
2.3. Árboles de Juego	5
2.4. Estados de Información	6
2.5. Minimax	6
2.6. Monte Carlo Tree Search	6
2.7. Information Set Monte Carlo Tree Search	7
2.8. Counterfactual Regret Minimization	7
3. Ambientes	8
3.1. Tic-Tac-Toe	8
3.2. Nocco Nocco	9
3.3. Kuhn Poker	9
4. Implementación	10
4.1. Interfaz de Juegos Alternados	10
4.2. Agente Aleatorio	10
4.3. Minimax	10
4.4. MCTS	10
4.5. ISMCTS	11
4.6. CFR	11
5. Experimentos	12
5.1. Tic-Tac-Toe	12
5.2. Nocco Nocco	12

5.3. Kuhn Poker	12
6. Conclusión	13

1. Introducción

Los sistemas multiagente permiten modelar problemas donde varios agentes toman decisiones dentro de un ambiente compartido [5, 4]. En el Obligatorio 2 se trabaja con juegos alternados, es decir, ambientes en los que los agentes actúan por turnos y el estado del juego evoluciona luego de cada acción. Este tipo de interacción aparece en juegos de tablero, juegos de cartas y problemas secuenciales de decisión estratégica.

A diferencia de los juegos simultáneos, en los juegos alternados cada decisión puede depender del historial previo. Esto permite estudiar algoritmos de búsqueda, planificación y aprendizaje en juegos extensivos, tanto con información perfecta como con información imperfecta. En particular, este trabajo considera ambientes como Tic-Tac-Toe, Nocca Nocca y Kuhn Poker.

El objetivo del trabajo es implementar y analizar agentes capaces de jugar en estos ambientes. Para ello se desarrollaron agentes basados en Minimax, Monte Carlo Tree Search (MCTS), Information Set Monte Carlo Tree Search (ISMCTS), Counterfactual Regret Minimization (CFR) y un agente aleatorio de referencia.

El código del obligatorio se organiza alrededor de una abstracción común para juegos alternados, compatible con la interfaz de PettingZoo. Cada ambiente define acciones disponibles, observaciones, transiciones, recompensas y condiciones de finalización, mientras que los agentes consumen esa interfaz para seleccionar acciones.

El informe se organiza de la siguiente manera. Primero se presenta el marco teórico necesario para comprender juegos alternados, búsqueda adversarial y juegos con información imperfecta. Luego se describen los algoritmos implementados y los ambientes utilizados. Finalmente, se deja planteada la estructura de experimentos a completar con los resultados obtenidos.

2. Marco Teórico

2.1. Juegos Alternados

Un juego alternado es un juego secuencial en el que, en cada instante, un único agente tiene el turno de actuar. Luego de seleccionar una acción, el ambiente actualiza su estado, puede entregar recompensas y selecciona el siguiente agente. Esta dinámica se puede representar mediante un árbol de juego, donde los nodos corresponden a estados y las aristas a acciones.

En juegos de información perfecta, todos los agentes observan completamente el estado relevante del juego. Tic-Tac-Toe y Nocka Nocka pertenecen a esta familia. En juegos de información imperfecta, los agentes solo observan una parte del estado real. Kuhn Poker es un ejemplo clásico, porque cada jugador conoce su carta, pero no la carta del oponente.

2.2. Juegos de Suma Cero

Muchos juegos alternados competitivos se modelan como juegos de suma cero. En estos juegos, la ganancia de un agente implica una pérdida equivalente para el otro. Esta propiedad permite razonar en términos de maximización para el agente propio y minimización para el adversario.

2.3. Árboles de Juego

El árbol de juego representa todas las posibles continuaciones desde un estado inicial. Cada nodo contiene un estado, el agente que debe actuar y las acciones legales disponibles. Los nodos terminales representan estados finales y contienen las recompensas para cada agente.

En juegos pequeños, el árbol completo puede recorrerse de forma exhaustiva. Sin embargo, en juegos con mayor factor de ramificación o profundidad, recorrer todo el árbol resulta costoso, por lo que se usan heurísticas, límites de profundidad o métodos de muestreo.

2.4. Estados de Información

En juegos con información imperfecta, un agente no siempre puede distinguir entre todos los estados reales posibles. El conjunto de estados compatibles con lo que el agente observó se denomina estado de información. Los algoritmos para estos juegos deben decidir en función de ese conjunto, no únicamente de un estado real completamente observable.

Kuhn Poker introduce esta dificultad de forma mínima: cada jugador observa su carta privada y el historial público de apuestas, pero desconoce la carta privada del rival. Por eso resulta adecuado para evaluar algoritmos como ISMCTS y CFR.

2.5. Minimax

Minimax es un algoritmo clásico de búsqueda adversarial para juegos de dos jugadores, alternados, deterministas y de suma cero. La idea central es elegir la acción que maximiza la utilidad del agente propio, suponiendo que el oponente elegirá acciones que minimizan dicha utilidad.

La recurrencia puede expresarse como:

$$V(s) = \begin{cases} U(s), & \text{si } s \text{ es terminal} \\ \max_{a \in A(s)} V(T(s, a)), & \text{si actúa el agente propio} \\ \min_{a \in A(s)} V(T(s, a)), & \text{si actúa el oponente} \end{cases}$$

Cuando el árbol es demasiado grande, se puede limitar la profundidad de búsqueda y utilizar una función de evaluación heurística para estimar el valor de estados no terminales.

2.6. Monte Carlo Tree Search

Monte Carlo Tree Search (MCTS) es un algoritmo de búsqueda que estima el valor de las acciones mediante simulaciones [1]. Cada iteración recorre cuatro etapas: selección, expansión, simu-

lación y retropropagación. La selección suele guiarse mediante UCB/UCT, balanceando explotación de acciones prometedoras y exploración de acciones poco visitadas.

MCTS resulta útil cuando el espacio de estados es grande y no se desea recorrer el árbol completo. En este obligatorio se usa como agente general para juegos alternados de información perfecta, realizando rollouts aleatorios desde estados expandidos.

2.7. Information Set Monte Carlo Tree Search

Information Set Monte Carlo Tree Search (ISMCTS) adapta MCTS a juegos con información imperfecta. En lugar de construir el árbol sobre estados reales completamente conocidos, el algoritmo opera sobre estados de información. En cada simulación se samplea una determinización compatible con lo observado por el agente y se ejecuta una búsqueda sobre ese posible estado.

Esta aproximación permite aplicar búsqueda Monte Carlo en juegos como Kuhn Poker, donde el agente debe razonar sobre cartas ocultas y decisiones del oponente sin conocer completamente el estado real.

2.8. Counterfactual Regret Minimization

Counterfactual Regret Minimization (CFR) es un algoritmo utilizado para aproximar estrategias en juegos extensivos con información imperfecta [3]. El algoritmo actualiza regrets por estado de información, comparando la utilidad esperada de las acciones disponibles contra la utilidad de la política actual.

La política se deriva de los regrets positivos acumulados mediante regret matching. Con suficientes iteraciones, en ciertos juegos de dos jugadores y suma cero, la estrategia promedio puede aproximarse a un equilibrio de Nash.

3. Ambientes

3.1. Tic-Tac-Toe

Tic-Tac-Toe es un juego alternado, determinista, de suma cero e información perfecta para dos jugadores. El tablero tiene dimensión 3×3 y los jugadores colocan marcas por turnos. El objetivo es completar una fila, columna o diagonal con tres marcas propias antes que el oponente.

En cada turno, el agente activo elige una acción entre las posiciones vacías del tablero. Como el tablero tiene nueve celdas, al inicio existen nueve acciones posibles. Luego de cada movimiento, el conjunto de acciones legales se reduce a las casillas todavía libres. La observación del agente contiene el estado completo del tablero, por lo que no existe información oculta ni incertidumbre sobre las jugadas anteriores.

Los valores terminales posibles para un agente son:

$$R_i = \begin{cases} 1, & \text{si el agente gana la partida} \\ -1, & \text{si el agente pierde la partida} \\ 0, & \text{si la partida termina en empate} \end{cases}$$

Como es un juego de suma cero, la recompensa del oponente es la opuesta cuando hay ganador. Si la partida termina en empate, ambos agentes reciben recompensa cero. En la implementación utilizada, estas recompensas se obtienen desde el ambiente al alcanzar un estado terminal.

Desde el punto de vista teórico, Tic-Tac-Toe es un juego resuelto. Si ambos jugadores actúan de manera óptima, el resultado esperado es empate. Por lo tanto, el valor del estado inicial bajo juego perfecto es:

$$V(s_0) = 0$$

Esto significa que ningún jugador puede forzar una victoria si el otro responde óptimamente. En términos de equilibrio, existe una estrategia óptima para cada jugador tal que ninguno puede mejorar unilateralmente su resultado esperado. Ese perfil de estrategias constituye un equilibrio de Nash en estrategias puras para el juego extensivo: frente a juego óptimo del rival, desviarse no permite obtener una recompensa mayor.

3.2. Nocca Nocca

Nocca Nocca es un juego de tablero alternado para dos jugadores. Cada jugador controla piezas sobre una grilla y puede moverlas de acuerdo con reglas de legalidad definidas por el tablero. El ambiente incorpora control de turnos, acciones legales, detección de finalización y recompensas según el ganador.

Este ambiente tiene un espacio de acciones más amplio que Tic-Tac-Toe y permite estudiar el comportamiento de algoritmos de búsqueda en un juego con mayor complejidad combinatoria.

3.3. Kuhn Poker

Kuhn Poker es un juego de cartas simplificado de dos jugadores y suma cero [2]. Se utiliza una baraja de tres cartas y cada jugador recibe una carta privada. Los jugadores alternan acciones de pasar o apostar, y el resultado depende del historial de apuestas y de las cartas privadas.

El valor del ambiente radica en que es un juego pequeño pero con información imperfecta. Cada agente conoce su propia carta y el historial público, pero no conoce la carta del oponente. Por este motivo, Kuhn Poker es especialmente adecuado para evaluar ISMCTS y CFR.

4. Implementación

4.1. Interfaz de Juegos Alternados

Los ambientes del obligatorio comparten una interfaz base llamada **AlternatingGame**. Esta abstracción define operaciones comunes como **reset**, **step**, **observe**, **available_actions**, **reward**, **terminated**, **truncated**, **game_over** y **clone**.

La operación **clone** es central para los algoritmos de búsqueda, ya que permite simular acciones futuras sin modificar el estado real del juego. Esta separación entre estado real y estado simulado es utilizada por Minimax, MCTS e ISMCTS.

4.2. Agente Aleatorio

El agente aleatorio selecciona uniformemente una acción entre las acciones legales disponibles. Se utiliza como línea base para comparar el desempeño de agentes más sofisticados.

4.3. Minimax

El agente Minimax recorre el árbol de juego desde el estado actual. Cuando el turno pertenece al agente propio, selecciona la acción que maximiza el valor esperado; cuando actúa el oponente, asume que este minimizará dicho valor. La implementación permite limitar la profundidad y utilizar una función de evaluación cuando no se alcanza un estado terminal.

4.4. MCTS

El agente MCTS construye un árbol de búsqueda a partir del estado actual. En cada simulación selecciona nodos mediante UCT, expande acciones no exploradas, ejecuta rollouts aleatorios y retropropaga las recompensas obtenidas. La acción final se elige a partir del valor medio estimado de los hijos de la raíz.

4.5. ISMCTS

El agente ISMCTS extiende la lógica de MCTS para trabajar con estados de información. Para cada simulación se obtiene un estado compatible con lo observado por el agente. Si el ambiente implementa una forma específica de samplear desde el conjunto de información, el agente la utiliza; en caso contrario, recurre a una copia del estado actual.

4.6. CFR

El agente CFR mantiene nodos asociados a observaciones o estados de información. En cada iteración recorre recursivamente el juego, calcula utilidades por acción y actualiza regrets contrafactuales. La política aprendida surge del promedio de las políticas generadas por regret matching.

5. Experimentos

5.1. Tic-Tac-Toe

Random vs Random

Minimax vs Random

MCTS vs Random

MCTS vs Minimax

ISMCTS vs MCTS

5.2. Nocca Nocca

Random vs Random

MCTS vs Random

ISMCTS vs Random

MCTS vs ISMCTS

5.3. Kuhn Poker

Random vs Random

CFR vs Random

ISMCTS vs Random

CFR vs ISMCTS

Política aprendida por CFR

Análisis de información imperfecta

6. Conclusión

Este obligatorio permite estudiar problemas de decisión secuencial multiagente mediante juegos alternados. A partir de una interfaz común para ambientes, se implementaron agentes representativos de distintas familias: búsqueda adversarial exhaustiva, búsqueda Monte Carlo, búsqueda sobre estados de información y minimización de regret.

Tic-Tac-Toe ofrece un punto de partida controlado para verificar el funcionamiento de los algoritmos en información perfecta. Nocca Nocca introduce un espacio de acciones más complejo, donde la eficiencia de la búsqueda se vuelve más relevante. Kuhn Poker agrega información imperfecta y permite observar la necesidad de razonar sobre estados compatibles con la observación del agente.

En conjunto, los ambientes y agentes implementados muestran cómo la estructura del juego condiciona la elección del algoritmo. Minimax resulta natural para juegos pequeños de información perfecta, MCTS permite escalar mediante simulaciones, ISMCTS adapta la búsqueda a información imperfecta y CFR permite aprender políticas promedio en juegos extensivos.

Referencias

- [1] Levente Kocsis and Csaba Szepesvari. Bandit based monte-carlo planning. *European Conference on Machine Learning*, 2006.
- [2] Harold W. Kuhn. A simplified two-person poker. In *Contributions to the Theory of Games*. Princeton University Press, 1950.
- [3] Marc Lanctot, Kevin Waugh, Martin Zinkevich, and Michael Bowling. Monte carlo sampling for regret minimization in extensive games. *Advances in Neural Information Processing Systems*, 2009.
- [4] Stuart Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*. Pearson, 2021.

- [5] Yoav Shoham and Kevin Leyton-Brown. *Multiagent Systems: Algorithmic, Game-Theoretic, and Logical Foundations*. Cambridge University Press, 2008.